

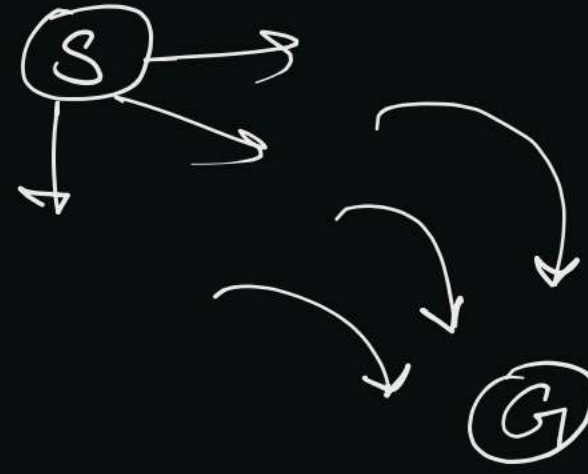
Informed search (Heuristic Search) :-

- ↳ These search algorithms use some domain-specific hints about the location of the goal(s).
- ↳ Can find solutions more efficiently (quickly) than an uninformed one.
- ↳ These hints/estimates are calculated by f^n :

$h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state.

Heuristic :- $\rightarrow$ find or discover

$\hookrightarrow$ Using our knowledge of the problem to consider some successors of the current state.



$\hookrightarrow$ Pruning the state space, for gaining speed, but perhaps missing the solution.

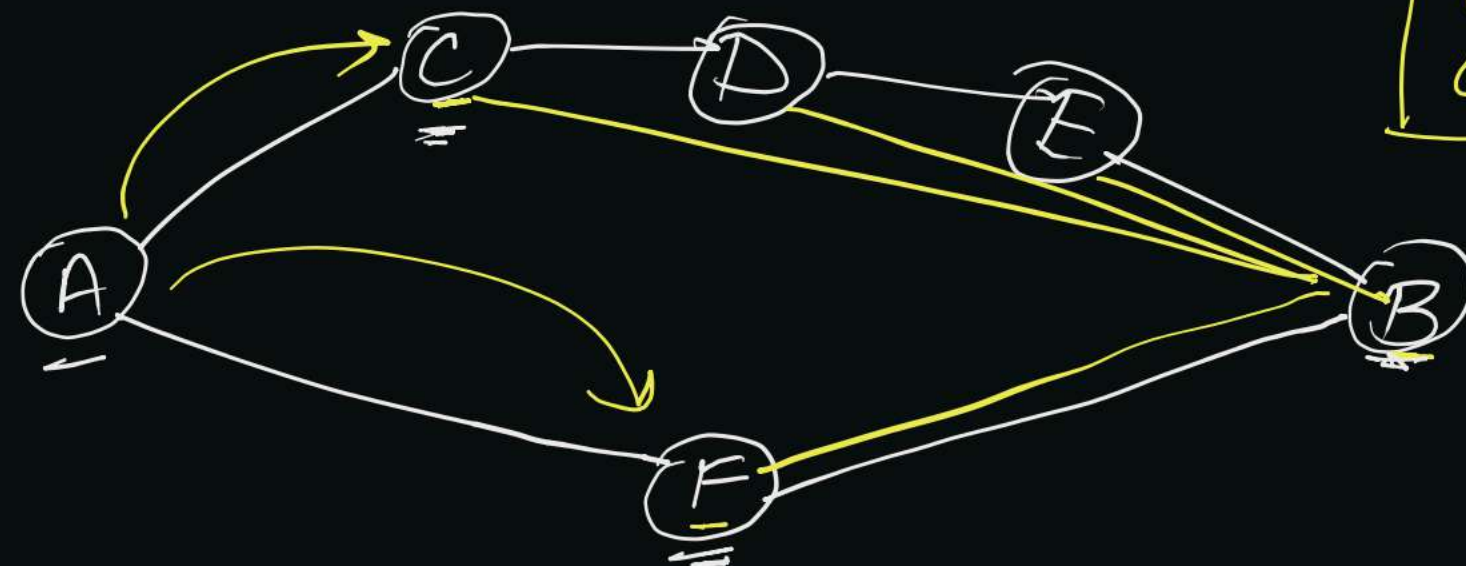
$\hookrightarrow$ We must be able to rank the successors of a node.

⑦ $\rightarrow$ heuristic $f^n \rightarrow$ estimated cost to the goal of all successors.

eg. If we want to reach city B from city A,
Euclidean distance $\rightarrow$ route-finding problem.

$A \rightarrow C$

$A \rightarrow F$



admissibility
consistency

Best-first search :- (priority queue will be used)

↳ Using an evaluation function to decide which adjacent node is most promising & then explore.

pseudo algorithm :-

```
q = priority-queue( );  
→ q.enqueue(start, 0)  
→ while (q is not empty)  
{  
→ t = q.dequeue( );  
✓ if (t == g) → return t ;  
→ else  
{  
for each t' in expand(t, cost-function)  
{  
→ if [(t' is not visited) or (t'.cost < visited[t'].cost)]  
{  
    visited ← t' ;  
    q.enqueue(t') ;  
}  
}  
}  
}
```

(10)

Current cost of t' getting from expand

previous best cost -

```
expand(node, cost-function)
```

This cost function is responsible to provide the ranking

Choose the next best.

```
{
```

```
  nodes = []
```

```
  for all neighbours n' of node
```

```
  {
```

```
    n'.cost = cost-function(n, n');
```

```
    nodes.append(n');
```

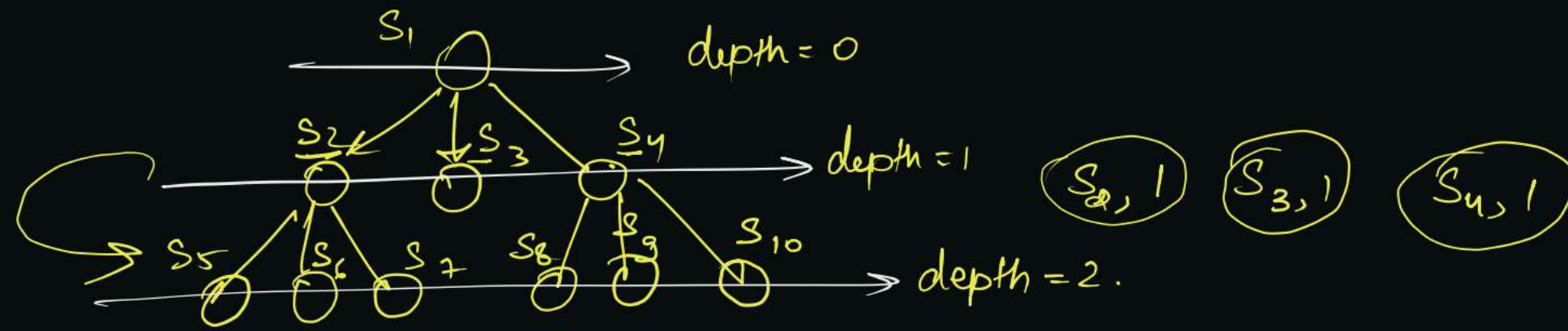
```
  }
```

```
  return nodes;
```

```
}
```


Cost-function :-

1. cost-function = $\boxed{\text{depth}(n) + 1}$ } BFS.



$(s_1, 0)$	$(s_2, 1)$	$(s_3, 1)$	$(s_4, 1)$	$(s_5, 2)$	$(s_6, 2)$	
------------	------------	------------	------------	------------	------------	-------

$(2, 3, 4) \rightarrow$ all 9 combinations are equivalent.

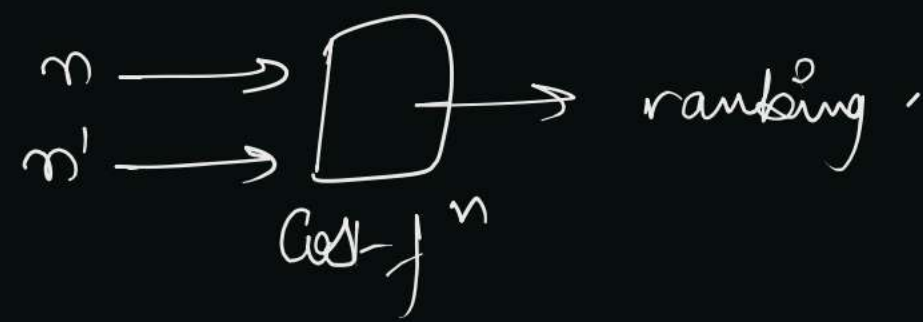
2. cost-function = (Sum of min. cost from start $\rightarrow n$) + next node's cost. } uniform Cost Search.

Best-first search

- BFS
- Uniform cost
- Greedy best-first
- A^*

} uninformed search techniques.

} informed search techniques.

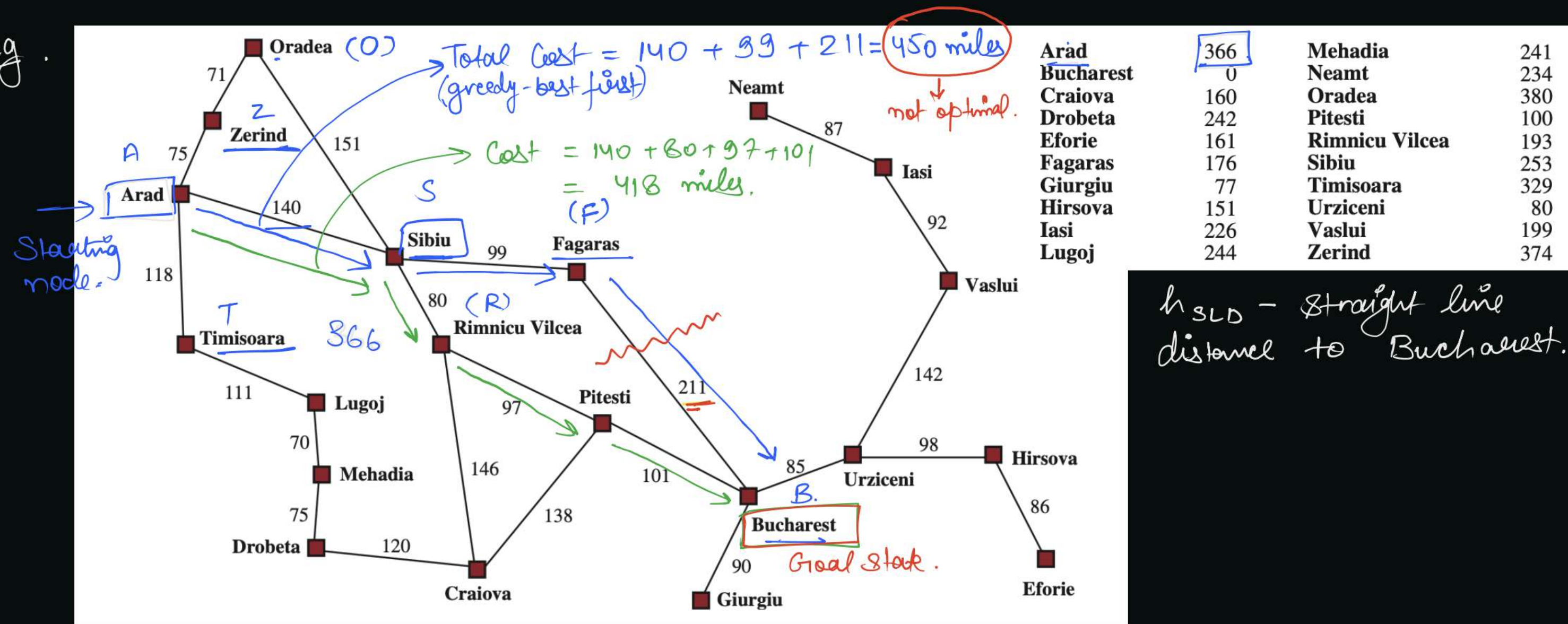


Greedy best first Search :-

↳ This search algorithm will expand the node that appears to be closest to the goal.

↳ Evaluation function, $f(n) = h(n)$ (heuristic function).
 $= \text{estimated cost}(n, \text{goal})$
 $= h_{SLD}(n)$ (straight line distance, $n \rightarrow \text{goal}$).

eg.

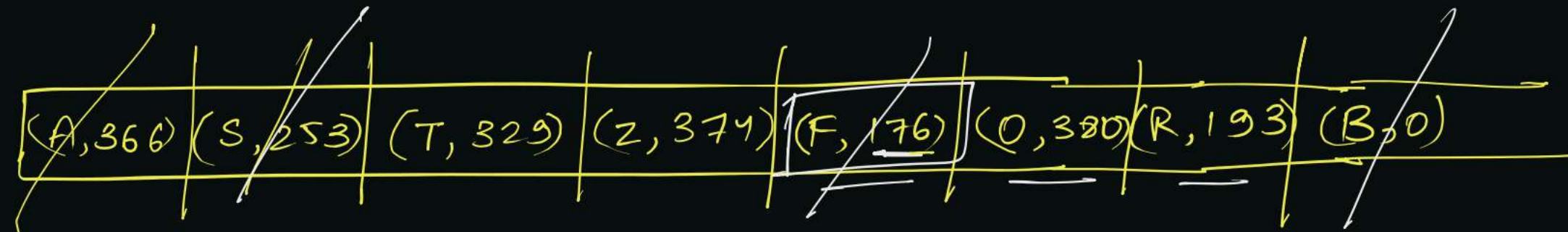
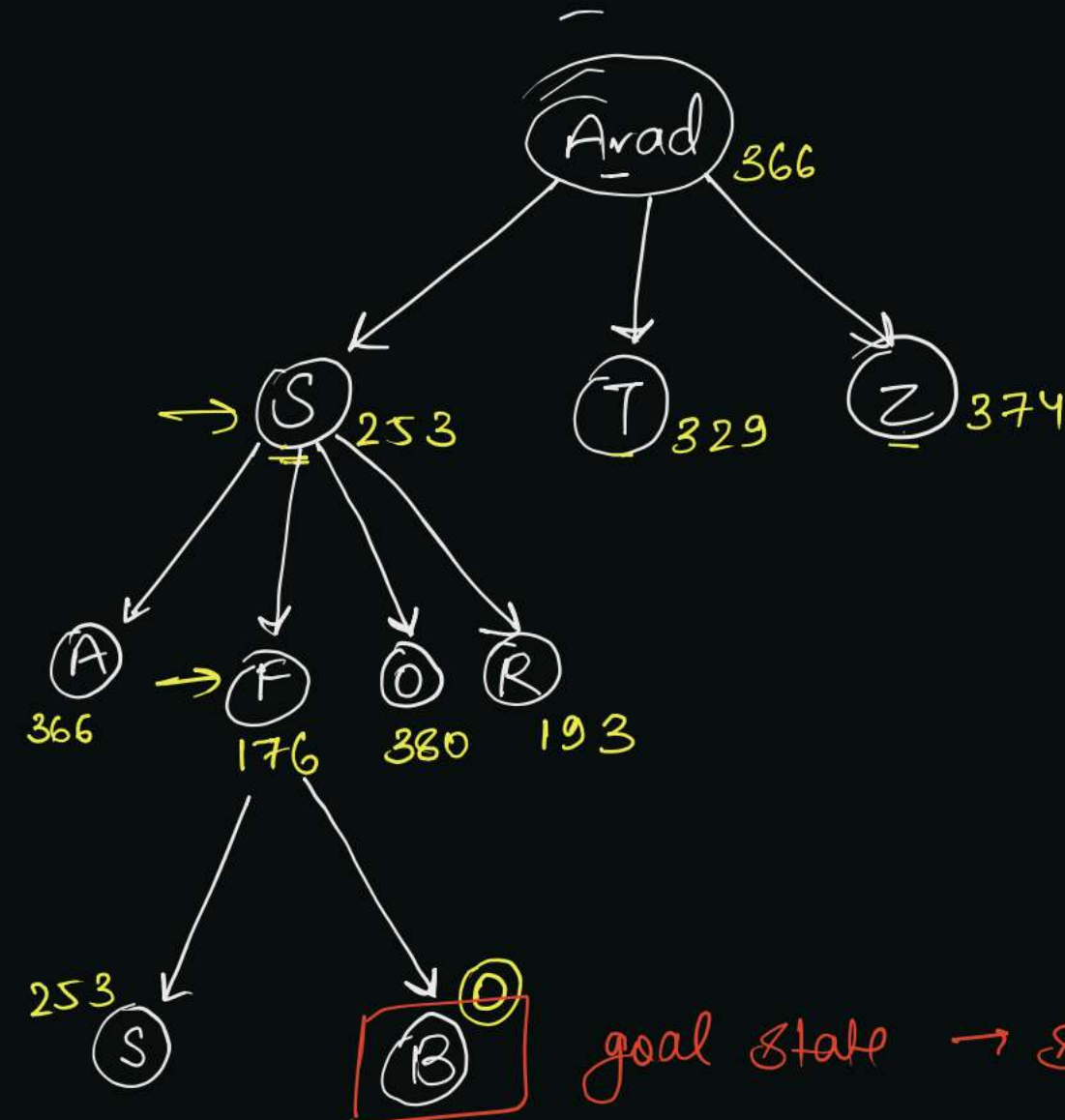


Road-map of Romania



greedy - best first
heuristic → Euclidean distance.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



A → S → F → B = 450 miles.

goal state → search stops.

Performance analysis :-

1. Complete :- no, because of the greedy nature & the heuristic.

2. Time :-

time complexity = $O(b^m)$ $\rightarrow$ it will improve if a good heuristic function is chosen. }

3. Space :-

space complexity = $O(b^m)$ $\rightarrow$ all nodes at depth m are stored in memory.

4. Optimal :- no (it's greedy).

$$b^0 + b^1 + b^2 + \dots + b^m$$

$\rightarrow O(b^m)$

